Tailored vs. Generic LLM Code Review: Related Work

Cory Rahman
George Mason University
Fairfax, Virginia, USA
crahman@gmu.edu

1 Related Work

1.1 Code Review Practices and Variations

Most feedback in code review is not about finding bugs. Bacchelli and Bird [2] found that defects account for only ∼14% of review comments, with the majority addressing non-defect concerns such as convention enforcement and alternative solutions. Czerwonka et al. [4] corroborate this: only ∼15% of review comments indicate possible defects, while ∼50% are maintenance-related. Static analysis tools cover only a fraction of this broader feedback. Singh et al. [15] showed that static analysis tool PMD overlapped with only ∼16% of reviewer comments. The remaining ∼84% covers software design, conventions, and maintainability.

What reviewers emphasize varies across projects. Rigby and Bird [12] found that while review process parameters converge (two reviewers, small changes, quick turnaround), the substance of review feedback diverges, with knowledge sharing during review increasing distinct files known by 66% to 150% depending on the project. Sadowski et al. [13] describe Google's "readability" certification, which requires language-specific style expertise before a developer can approve changes, showing how review norms can become institutionalized at the organizational level.

1.2 LLM-Based Automated Code Review

Automating code review with neural models has progressed from fine-tuned neural machine translation to prompted LLMs. Tufano et al. [18] pioneered mining Gerrit code review repositories to train neural models on code changes, achieving 9–17% perfect predictions on small methods via NMT. Li et al. [8] extended this paradigm from code refinement to review comment generation with CodeReviewer, pre-trained on 1,000+ repositories and establishing the standard benchmark. Pornprasit and Tantithamthavorn [11] systematically compared 12 variations of fine-tuning and prompting across GPT-3.5 and Magicoder: fine-tuned GPT-3.5 achieved 73–74% higher exact match than prior approaches, and when fine-tuning data was unavailable, few-shot learning achieved 46–659% higher exact match than zero-shot. All prompts in their study were generic and project specificity was never varied. Guo et al. [5] found that ChatGPT with generic prompts surpassed the fine-tuned CodeReviewer (22.78 vs. 15.50 exact match, 76.44 vs. 62.88 BLEU-4), suggesting that project-specific prompt optimization could push frontier LLMs further still.

Industrial deployments show that LLM code review works at scale, but generic approaches hit a ceiling. Sun et al. [16] deployed BitsAI-CR at ByteDance with 219 org-specific review rules, achieving 75.0% precision across 12,000+ weekly active users. Cihan et al. [3] studied 238 practitioners across 10 projects and found that while 73.8% of automated comments were resolved, effectiveness varied significantly by project, with practitioners citing irrelevant comments as a drawback. Tantithamthavorn et al. [17] deployed RovoDev at Atlassian across 1,900+ repositories with generic zero-shot prompting, achieving 38.7% code resolution rate; they explicitly called for "further research into advanced context augmentation techniques." Two approaches adapt reviews to organizational context but have limitations: Nashaat and Miller's CodeMentor [9] uses a three-phase fine-tuning pipeline (SFT, self-instruct, RLHF) requiring gradient access and domain experts, and Zhang et al.'s LAURA [19] retrieves review exemplars for prompt augmentation but never ablates same-project vs. cross-project retrieval. No existing work performs a controlled comparison of generic vs. project-specific prompts for LLM code review.

1.3 Project-Specific Adaptation

Software engineering tasks benefit from project-specific context. Defect prediction tends to fail to generalize to other projects, as shown by Zimmermann et al. [21]. Only 3.4% of 622 cross-project defect prediction combinations yielded acceptable performance. Furthermore, Shimagaki et al. [14] showed that project-specific adaptation was necessary in order to successfully measure code review quality; therefore, generating review feedback would likely also benefit from project-specific adaptation. In-context learning can overcome transfer failure, especially with project-specific examples. Ahmed and Devanbu [1] used GPT Codex to summarize code, showing that few-shot in-context learning with just 10 examples beat models fine-tuned on 24K–251K examples. They also showed that same-project examples yielded a 12.56% score improvement over cross-project examples and identified "highly project-specific" phenomena such as identifier names, APIs, terminology, and coding patterns. Nashid et al. [10] showed that using few-shot prompting to provide project-specific context outperformed fine-tuned models by 11% on code tasks. These previous works motivate project-specific prompt optimization for code review.

1.4 Automated Prompt Optimization

Prompt optimization is a mature technique with proven gains in NLP and software engineering tasks. APE [20] matched or surpassed human-written prompts on all 24 Instruction Induction tasks, slightly outperforming humans in aggregate. DSPy [7] extended this to multi-stage pipelines, outperforming standard few-shot on NLP benchmarks by 25%+ with GPT-3.5 and 65%+ with llama2-13b-chat. These techniques transfer to software engineering: Ji et al. [6] applied automated prompt optimization to SWE tasks, achieving approximately 28% improvement on translation, 58% on summarization, and 84% on API recommendation; notably, code review was untested.

1.5 Novelty

Our work differs from prior approaches in two ways. First, we operate entirely at the prompt level, requiring no model modification. CodeMentor [9], the most comprehensive org-specific code review system, requires supervised fine-tuning, self-instruct data augmentation, and RLHF with domain experts. Our approach works with any model accessible via API, including closed-source frontier models, making it practical for teams without training infrastructure. Second, we perform the first controlled comparison of generic vs. project-specific prompts for LLM code review. BitsAI-CR [16] compares org-specific rules against no rules but confounds prompt specificity with architecture by simultaneously changing the pipeline, fine-tuning, and rules. Cihan et al. [3] and RovoDev [17] observe per-project variation but deploy the same generic tool uniformly. No prior work isolates prompt specificity as the sole variable, using the same model and architecture for both conditions across multiple projects.

References

[1] Toufique Ahmed and Premkumar Devanbu. 2022. Few-Shot Training LLMs for Project-Specific Code-Summarization. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. doi:10.1145/3551349.3559555

[2] Alberto Bacchelli and Christian Bird. 2013. Expectations, Outcomes, and Challenges of Modern Code Review. In *Proceedings of the 35th International Conference on Software Engineering (ICSE)*. 712–721. doi:10.1109/ICSE.2013.6606617

[3] Umut Cihan et al. 2025. Automated Code Review in Practice. In *Proceedings of the 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*.

[4] Jacek Czerwonka, Michaela Greiler, and Jack Tilford. 2015. Code Reviews Do Not Find Bugs: How the Current Code Review Best Practice Slows Us Down. In *Proceedings of the 37th International Conference on Software Engineering (ICSE)*. 27–28. doi:10.1109/ICSE.2015.131

[5] Shuzheng Guo, Tao Zhang, Wenchao Che, and Ting Liu. 2024. Exploring the Potential of ChatGPT in Automated Code Refinement: An Empirical Study. In *Proceedings of the 46th International Conference on Software Engineering (ICSE)*. 1–13.

[6] Jie Ji, Bin Xu, and Xiaofei Wang. 2025. Automated Prompt Generation for Code Intelligence. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), Industry Showcase*.

[7] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*.

[8] Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu, and Neel Sundaresan. 2022. Automating Code Review Activities by Large-Scale Pre-Training. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. doi:10.1145/3540250.3549081

[9] Mona Nashaat and John Miller. 2024. Towards Efficient Fine-Tuning of Language Models With Organizational Data for Automated Software Review. *IEEE Transactions on Software Engineering* 50, 9 (2024), 2267–2285. doi:10.1109/TSE.2024.3428324

[10] Noor Nashid, Mifta F. Sintaha, and Ali Mesbah. 2023. Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning. In *Proceedings of the 45th International Conference on Software Engineering (ICSE)*. 2450–2462. doi:10.1109/ICSE48619.2023.00205

[11] Chanathip Pornprasit and Chakkrit Tantithamthavorn. 2024. Fine-Tuning and Prompt Engineering for Large Language Models-based Code Review Automation. *Information and Software Technology* 175 (2024), 107523. doi:10.1016/j.infsof.2024.107523

[12] Peter C. Rigby and Christian Bird. 2013. Convergent Contemporary Software Peer Review Practices. In *Proceedings of the 9th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 202–212. doi:10.1145/2491411.2491444

[13] Caitlin Sadowski, Emma Söderberg, Luke Church, Michal Sipko, and Alberto Bacchelli. 2018. Modern Code Review: A Case Study at Google. In *Proceedings of the 40th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. 181–190. doi:10.1145/3183519.3183525

[14] Junji Shimagaki, Yasutaka Kamei, Shane McIntosh, Ahmed E. Hassan, and Naoyasu Ubayashi. 2016. A Study of the Quality-Impacting Practices of Modern Code Review at Sony Mobile. In *Proceedings of the 38th International Conference on Software Engineering Companion (ICSE-C)*. 212–221. doi:10.1145/2889160.2889190

[15] Devarshi Singh, Varun Ramachandra Sekar, Kathryn T. Stolee, and Brittany Johnson. 2017. Evaluating How Static Analysis Tools Can Reduce Code Review Effort. In *Proceedings of the 2017 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. 101–105. doi:10.1109/VLHCC.2017.8103456

[16] Tao Sun et al. 2025. BitsAI-CR: Automated Code Review via LLM in Practice. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE), Industry Track*.

[17] Kla Tantithamthavorn, Yaotian Zou, Andy Wong, Michael Gupta, Zhe Wang, Mike Buller, Ryan Jiang, Matthew Watson, Minwoo Jeong, Kun Chen, and Ming Wu. 2026. RovoDev Code Reviewer: A Large-Scale Online Evaluation of LLM-based Code Review Automation at Atlassian. In *Proceedings of the 48th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. arXiv:2601.01129.

[18] Michele Tufano, Jevgenija Pantiuchina, Cody Watson, Gabriele Bavota, and Denys Poshyvanyk. 2019. On Learning Meaningful Code Changes Via Neural Machine Translation. In *Proceedings of the 41st International Conference on Software Engineering (ICSE)*. 25–36. doi:10.1109/ICSE.2019.00021

[19] Yongkun Zhang, Yichi Zhang, Zhiyong Sun, Yanlin Jiang, and Hui Liu. 2025. LAURA: Enhancing Code Review Generation with Context-Enriched Retrieval-Augmented LLM. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

[20] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large Language Models Are Human-Level Prompt Engineers. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*.

[21] Thomas Zimmermann, Nachiappan Nagappan, Harald Gall, Emanuel Giger, and Brendan Murphy. 2009. Cross-Project Defect Prediction: A Large Scale Experiment on Data vs. Domain vs. Process. In *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE)*.